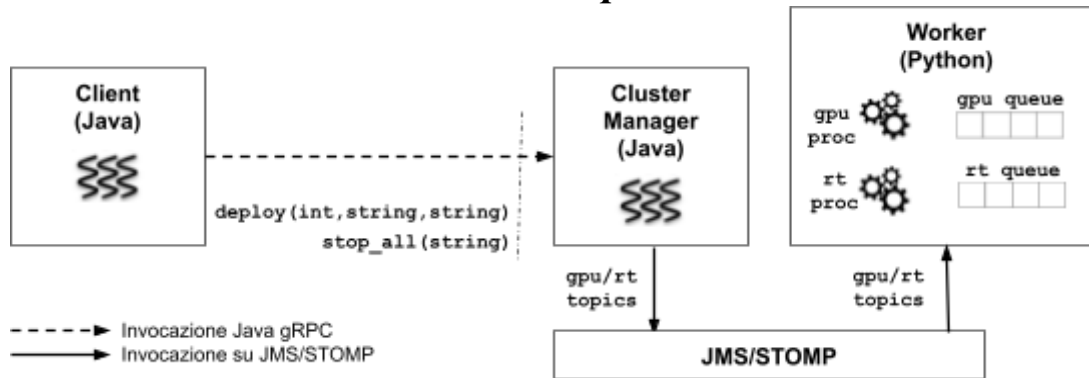


Università degli Studi di Napoli Federico II
Esame di Advanced Computer Programming
Proff. De Simone, Della Corte

Prova pratica del giorno 12/09/2025
Durata della prova: 120 minuti

Testo della prova



Il candidato implementi un sistema distribuito in **Java** e **Python** per l'avvio e la terminazione di task remoti basato su **Java gRPC** e **JMS/STOMP**. Il sistema è caratterizzato dai seguenti componenti.

Client. Componente **Java** multithread che funge da client gRPC verso il **Cluster Manager**. Il Client genera richieste di *deploy* e *stop_all* verso il Cluster Manager. L'invio di una richiesta *deploy* consiste nell'invocazione del metodo `void deploy(int, string, string)`. La richiesta è caratterizzata da 1) **id** (*int*), ossia l'ID univoco associato al task; 2) **name** (*string*), ossia il nome associato al task; 3) **type** (*string*), ossia la tipologia di task (gpu-bound o real-time). L'invio di una richiesta *stop_all* consiste nell'invocazione del metodo `void stop_all(string)`. La richiesta è caratterizzata dal campo **type** (*string*), ossia la tipologia di task (gpu-bound o real-time). Il client crea 5 thread: ogni thread invoca 4 volte il metodo *depoly* (attendendo 1 secondo tra le invocazioni) ed una volta il metodo *stop_all*. Per ciascuna richiesta, *type* è generato in maniera casuale, scegliendo una tra le tipologie di task (gpu-bound o real-time), come anche il valore dei campi *id* e *name*.

Cluster Manager. E' un server **Java** gRPC e client JMS, che riceve le richieste da **Client**. Per ogni invocazione di *deploy* o *stop_all* il manager avvia un thread che verifica il campo *type* ed invia un `TextMessage` sul topic *gpu* se il tipo è *gpu-bound* o su quello *rt* se invece il tipo è *real-time*. Nel caso di *deploy*, il `TextMessage` dovrà contenere la concatenazione di operazione-id-name, e.g., *deploy-10-rtcontrol*. Nel caso di *stop_all* invece, il `TextMessage` conterrà solo l'operazione richiesta, i.e., *stop_all*.

Worker. E' un componente **Python** che gestisce due code (coda del modulo `multiprocess/multiprocessing`) di dimensione 5, una per ogni topic (*gpu* ed *rt*). All'avvio il worker crea 2 processi, uno per ogni topic. Ogni processo implementa la ricezione asincrona STOMP su uno dei topic. Alla ricezione di un messaggio sul proprio topic, il processo analizza il contenuto del messaggio e verifica l'operazione richiesta, cioè *deploy* o *stop_all*. Se l'operazione è *deploy*, il processo inserisce il contenuto del messaggio nella coda relativa al proprio topic. Se l'operazione è *stop_all*, il processo svuota la coda relativa al proprio topic. In entrambi i casi, il processo stampa a video l'operazione effettuata, indicando cosa è stato inserito o cosa è stato rimosso dalla coda.